

Pintos Project 3: Threads

담당 교수 : 김영재

조 / 조원 : 20211558 윤준서

개발 기간 : 11.1 - 11.7 (7일)

I. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.

1) Alarm Clock

시간이 지나지 않은 경우, thread를 BLOCKED 상태로 전환하여 차단시킨다. 이때 thread들을 관리하기 위해 큐를 사용하고, 시간이 지났을 경우 thread를 wake up 후 큐에 삽입한다.

2) Priority Scheduling

thread에 우선 순위를 부여하여, 새로운 thread의 우선 순위가 더 높다면 CPU를 양보하도록 한다. 이때, ready 큐에 머문 시간에 비례하여 우선 순위를 증가시키도록 aging 기법을 사용한다.

3) Advanced Scheduler

MLFQ를 구현하여 multi priority 큐를 사용한다. 가장 높은 우선 순위 큐에서부터 thread를 선택해 실행시키는데, 각 우선 순위의 큐는 라운드 로빈 방식을 사용한다.

II. 개발 범위 및 내용

A. 개발 범위

- 아래 각 항목 개발의 필요성 또는 개발 시 기대되는 결과를 간략히 서술

1) Alarm Clock

RUNNING과 READY 상태만 존재했던 방식에서 BLOCKED 상태를 추가해 차단할 thread를 sleep시킨다. 이를 통해 RUNNING과 READY 상태를 반복하는 overhead를 줄일 수 있다.

2) Priority Scheduling

thread에 우선 순위를 부여하고, 우선 순위가 높은 thread를 우선 실행한다. 이때 aging 기법을 통해 ready 큐 내부 thread의 starvation을 방지한다. 이를 통해 평균 turnaround time을 줄일 수 있다.

3) Advanced Scheduler (추가구현을 한 경우)

소수점 연산을 구현 후, nice 값과 최근 cpu 사용 시간 등의 요소를 정의하고 이들을 통해 MLFQ를 구현한다. 이를 통해 단순 Priority Scheduling 방법보다 starvation과 turnaround time을 더 향상시킬 수 있다.

B. 개발 내용

- 아래 항목의 내용만 서술

1) Blocked 상태의 스레드를 어떻게 깨울 수 있는지 서술.

timer_sleep() 함수는 thread를 일정 시간 동안 BLOCKED 상태로 만든다. 이때 thread의 **wake up time = current ticks + sleep time** 이다. 해당 thread를 sleep list에 추가하고 BLOCKED 상태로 전환한다.

매 tick마다 발생하는 timer interrupt에서, sleep list를 순회하며 **current ticks >= wake up time**을 판단한다. 참일 경우, 해당 스레드의 BLOCKED를 해제하여 ready 큐에 삽입한다.

2) Ready list에 running thread보다 높은 priority를 가진 thread가 들어올 경우 priority scheduling에 따르면 어떻게 해야하는지 서술.

ready 큐에 더 높은 priority의 thread가 추가되면, 현재 실행 중인 thread는 yield하여 READY 상태로 전환하고, 추가된 thread는 READY 상태로 전환한다.

해당 과정은 thread_set_priority()로 priority를 변경하거나, 새로운 high-priority thread가 생성될 때 실행한다.

3) Advanced Scheduler에서 priority 계산에 필요한 각 요소를 서술. (추가구현을 한 경우)

우선 순위 계산식은 다음과 같다. **priority = PRI_MAX - (recent cpu / 4) - (nice * 2)**

PRI_MAX = 63 (최고 우선 순위), priority는 0~63, nice는 -20 ~ 20의 범위를 가진다. priority가 높고, nice가 작을 수록 우선 순위가 높다. 계산은 매 4 ticks마다 수행되며, CPU를 덜 사용한 thread에게 우선권을 주어 starvation을 방지한다.

최근 CPU 사용량을 뜻하는 recent cpu 계산식은 다음과 같다.

$$\text{recent cpu} = (2 * \text{load avg}) / (2 * \text{load avg} + 1) * \text{recent cpu} + \text{nice}$$

매 tick마다 실행 중인 thread의 recent cpu가 1씩 증가한다. 그리고 매 초마다 전체 thread의 recent cpu를 갱신한다.

전체 시스템의 READY 상태 thread의 평균 개수인 load avg 계산식은 다음과 같다.

$$\text{load avg} = (59 / 60) * \text{load avg} + (1 / 60) * \text{ready threads}$$

idle 상태인 thread는 제외하며, system 부팅 시 0으로 시작한다. 매 초 갱신한다.

III. 추진 일정 및 개발 방법

A. 추진 일정

- II. A. 개발 범위를 포함하여 구현 내용에 대한 일정 작성

11.1 - 11.2 : Alarm Clock 구현

11.3 - 11.4 : Priority Scheduling 구현

11.5 - 11.6 : BSD Scheduler 구현

11.7 : 보고서 작성

B. 개발 방법

- II. B.의 개발 내용을 구현하기 위해 각각에 대해 다음 사항들을 포함하여 설명

- 수정해야하는 소스코드

threads/init.c : **kernel command line**에서 mlfq를 추가한다.

devices/timer.c : **timer_sleep()**에서 일정 tick이 지나면 thread_sleep()을 호출하도록 한다.

- 수정하거나 추가해야 하는 자료구조

threads/synch.h : lock에 held_locks 리스트를 사용하기 위해 **list_elem held_elem**을 추가한다.

threads/synch.c : semaphore_elem에 우선 순위를 나타내는 **int priority**를 추가한다.

threads/thread.h : 현재 우선 순위, yield 진행 전의 우선 순위, 현재 tread가 보유하고 있는 **lock list**, 현재 thread가 대기 중인 lock, wake up에 필요한 tick 시간, nice, recent cpu를 추가한다.

- 수정하거나 추가해야 하는 함수

threads/fixed-point.h : 부동 소수점 연산을 수행하는 함수들을 새로운 헤더 파일을 통해 정의한다.

threads/synch.c : **sema_down()**, **lock_acquire()**, **lock_release()**를 수정하여 우선 순위가 높은 thread의 semaphore를 조정한다.

cond_priority_greater()를 추가하여 두 semaphore_elem의 우선 순위를 비교한다.

threads/thread.c :

thread_check_preemption()을 추가하여 우선 순위가 높은 thread에게 yield 한다.

thread_init()에서 추가한 thread의 요소들을 초기화한다.

thread_tick()에서 tick마다 recent cpu, load avg, thread의 우선 순위를 갱신한다.

thread_create()에서 새로 추가된 thread의 우선 순위가 높으면 yield 받도록 한다.

thread_set_priority()를 추가하여 실행중인 thread의 우선 순위를 갱신한다.

thread_set_nice()를 추가하여 thread의 nice 값을 설정한다.

thread_sleep()를 추가하여 현재 thread를 BLOCKED 상태로 전환한다.

thread_wake_up()를 추가하여 시간이 지난 thread를 wake up 시킨다.

thread_aging()을 통해 모든 thread의 우선 순위를 갱신, 순서대로 정렬한다.

mlfqs_update_load_avg()를 추가해 load avg를 갱신한다.

mlfqs_calculate_recent_cpu()를 추가해 1초마다 모든 thread의 recent cpu를 갱신한다.

mlfqs_calculate_priority()를 추가해 1초마다 모든 thread의 우선 순위를 갱신한다.

mlfqs_increment_recent_cpu()를 추가해 tick마다 현재 thread의 recent_cpu를 갱신한다.

mlfqs_recalculate_call()를 추가해 1초마다 모든 thread의 recent_cpu, 우선 순위, load avg를 갱신하는 함수들을 호출한다.

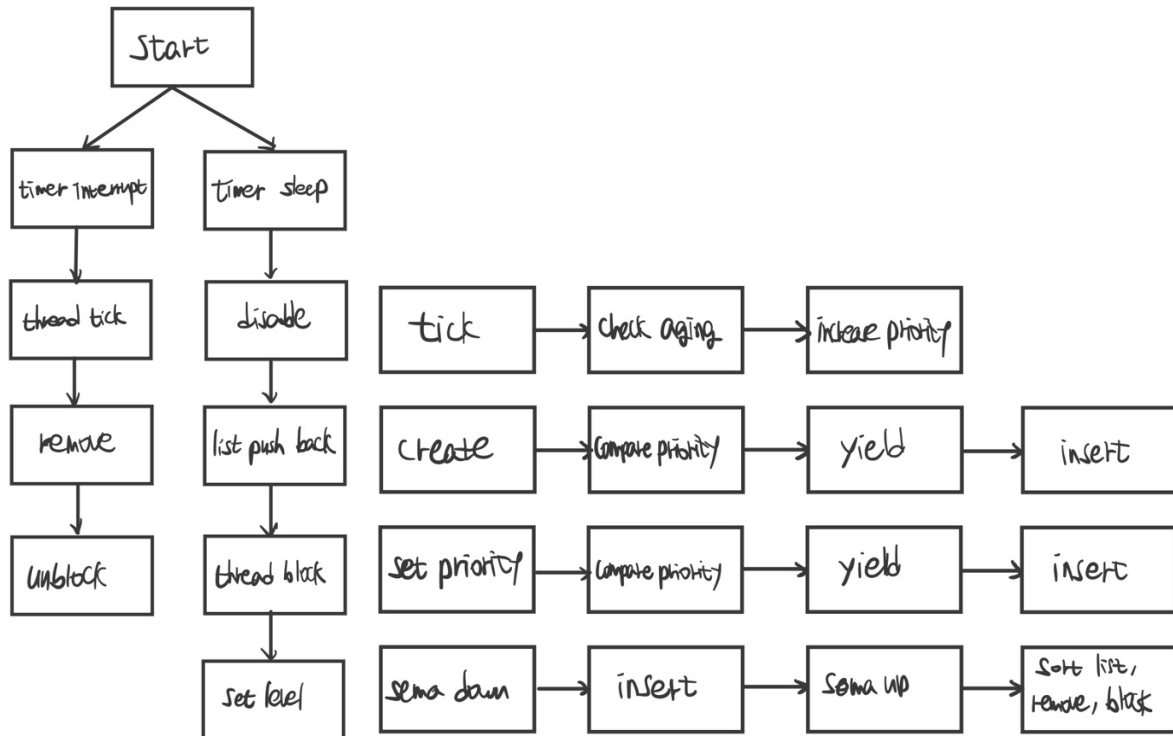
IV. 연구 결과

A. Flow Chart

- II. B. 개발 내용의 각 항목에 대하여 Flow Chart 작성
(추가구현에 대해서는 flow chart를 작성하지 않아도 됨)

1. Alarm Clock

2. Priority Scheduling



B. 제작 내용

- II. B. 개발 내용의 각 항목에 대하여 실질적으로 구현한 코드의 관점에서 작성 (구현 내용, 알고리즘 등을 명확히 서술할 것)
- ✓ 구현에 있어 Pintos에 내장된 라이브러리나 자체 제작한 함수를 사용한 경우 이에 대해서도 설명

```
#endif
" -rs=SEED          Set random number seed to SEED.\n"
" -mlfq             Use multi-level feedback queue scheduler.\n"
```

threads/init.c : **kernel command line**에서 mlfq를 추가한다. 이를 통해 파일 실행 시 일반 priority scheduler인지, MLFQ인지 구분할 수 있다.

```

#ifndef THREADS_FIXED_POINT_H
#define THREADS_FIXED_POINT_H

/* 고정 소수점 */
#define F (1 << 14)

/* 정수를 고정 소수점으로 변환 */
#define INT_TO_FP(n) ((n) * F)

/* 고정 소수점을 정수로 변환 (소수점 이하 버림) */
#define FP_TO_INT_TRUNC(x) ((x) / F)

/* 고정 소수점을 정수로 변환 (반올림) */
#define FP_TO_INT_ROUND(x) ((x) >= 0 ? ((x) + F / 2) / F : ((x) - F / 2) / F)

```

threads/fixed-point.h : 부동 소수점 연산을 수행하는 함수들을 새로운 헤더 파일을 통해 정의한다.

[devices/timer.c]

```

void timer_sleep (int64_t ticks) {
    if (ticks <= 0) return;

    int64_t start = timer_ticks();
    ASSERT (intr_get_level() == INTR_ON);

    thread_sleep(start + ticks);
}

```

thread가 0 이하의 tick 동안 sleep을 요청하면 return 한다.

wake up time을 계산하고 thread_sleep()을 호출한다.

[threads/synch.c]

```

void sema_down (struct semaphore *sema) {
    enum intr_level old_level;

    ASSERT (sema != NULL);
    ASSERT (!intr_context ());

    old_level = intr_disable ();
    while (sema->value == 0) {
        if (!thread_mlfqs)
            list_insert_ordered (&sema->waiters, &thread_current ()->elem,
                                thread_priority_greater, NULL);
        else
            list_push_back (&sema->waiters, &thread_current ()->elem);
        thread_block ();
    }
    sema->value--;
    intr_set_level (old_level);
}

```

interrupt를 끄고, semaphore가 0일 경우 현재 thread를 waiting list에 삽입한다. 이때 삽입의 방식은 MLFQ의 여부에 따라 달라진다. 이후 thread를 BLOCKED 상태로 전환 후 대기시킨다. 자원이 확보되면 (value > 0) 하나를 가져간다. 그리고 마지막으로 interrupt를 복원시켜 커널을 다시 작동시킨다.

```

static bool cond_priority_greater (const struct list_elem *a,
                                   const struct list_elem *b,
                                   void *aux UNUSED) {
    const struct semaphore_elem *sa = list_entry (a, struct semaphore_elem, elem);
    const struct semaphore_elem *sb = list_entry (b, struct semaphore_elem, elem);
    return sa->priority > sb->priority;
}

```

두 semaphore_elem의 우선 순위를 비교하는 함수를 추가했다.

```

if (lock->holder != NULL) {
    cur->waiting_on_lock = lock;

    if (cur->priority > lock->holder->priority)
        thread_donate_priority (lock->holder);
}

intr_set_level (old_level);
sema_down (&lock->semaphore);

old_level = intr_disable ();
cur->waiting_on_lock = NULL;
list_push_back (&cur->held_locks, &lock->held_elem);
lock->holder = cur;
intr_set_level (old_level);

```

lock_acquire()의 일부다. 다른 thread에서 이미 lock을 잡고 있을 경우, 현재 thread가 대기 중임을 명시한다. 그리고 현재 thread의 우선 순위가 lock holder보다 높으면, donation을 받는다. 이후 sema_down()을 통해 lock을 얻고, 받았음을 명시한다. 그리고 held_locks에 해당 lock을 추가하고, 주인을 현재 thread로 명시한다.

```

list_remove (&lock->held_elem);
lock->holder = NULL;

thread_recalculate_priority (cur);

struct thread *t = NULL;

if (!list_empty (&lock->semaphore.waiters)) {
    t = list_entry (list_pop_front (&lock->semaphore.waiters), struct thread, elem);
    thread_unblock (t);
}

lock->semaphore.value++;

if (t != NULL && t->priority > thread_current ()->priority) thread_yield ();

intr_set_level (old_level);

```

lock_release()의 일부다. held_locks에서 인자로 받은 lock을 제거하고, 주인을 NULL로 명시한다. 우선 순위를 다시 계산하고 해당 lock을 위해 대기 중인 thread가 있으면 해당 thread를 wake up 한다. 이때 깨운 thread가 현재 thread보다 우선 순위가 높으면 yield 한다.

[threads/thread.c]

```

void thread_check_preemption (void) {
    if (list_empty (&ready_list)) return;

    if (thread_current () == idle_thread || intr_context ()) return;

    struct thread *highest_ready = list_entry (list_front (&ready_list), struct thread, elem);
    if (thread_current ()->priority < highest_ready->priority) thread_yield ();
}

```

현재 thread로부터 우선 순위가 가장 높은 thread에게 cpu를 yield 한다.

```

void thread_set_priority (int new_priority) {
    if (thread_mlfqs) return;

    enum intr_level old_level = intr_disable ();

    struct thread *cur = thread_current ();
    cur->base_priority = new_priority;

    thread_recalculate_priority (cur);
    thread_check_preemption ();
    intr_set_level (old_level);
}

```

```

void thread_set_nice (int nice UNUSED) {
    struct thread *cur = thread_current ();
    cur->nice = nice;

    mlfqs_calculate_priority (cur);

    int highest_pri = PRI_MIN;
    for (int i = PRI_MAX; i > cur->priority; i--) {
        if (!list_empty (&ready_list[i])) {
            highest_pri = i;
            break;
        }
    }

    if (cur->priority < highest_pri) thread_yield ();
}

```

thread_set_priority()는 현재 thread의 우선 순위를 갱신한다 **thread_set_nice()**는 현재 thread의 nice 값을 부여 한다. 이때 우선 순위가 가장 높은 thread에게 yield 한다.


```

if (thread_mlfqs) {
    mlfqs_increment_recent_cpu ();

    if (timer_ticks () % TIMER_FREQ == 0) {
        mlfqs_update_load_avg ();

        struct list_elem *e;
        for (e = list_begin (&all_list); e != list_end (&all_list); e = list_next (e))
            mlfqs_calculate_recent_cpu (list_entry (e, struct thread, allelem));
    }

    if (timer_ticks () % TIME_SLICE == 0) {
        struct list_elem *e;
        for (e = list_begin (&all_list); e != list_end (&all_list); e = list_next (e))
            mlfqs_calculate_priority (list_entry (e, struct thread, allelem));
    }
}

if (++thread_ticks >= TIME_SLICE) intr_yield_on_return ();

thread_wake_up ();

if (thread_prior_aging == true) thread_aging ();

if (thread_mlfqs) {
    for (int i = PRI_MAX; i > thread_current ()->priority; i--) {
        if (!list_empty (&ready_list[i])) {
            intr_yield_on_return ();
            break;
        }
    }
}
else {
    if (!list_empty (&ready_list) &&
        thread_current ()->priority <
        list_entry (list_front (&ready_list), struct thread, elem)->priority)
        intr_yield_on_return ();
}
}

```

thread_tick()의 일부다. MLFQ일 경우, recent cpu와 load avg를 기반으로 우선 순위를 조정한다.

매 tick마다 현재 thread의 recent cpu를 1 증가시킨다. 그리고 1초마다 시스템 전체 load avg와 모든 thread의 recent cpu를 갱신한다.

매 4 tick마다 모든 thread의 우선 순위를 갱신한다. 그리고 현재 thread가 TIME_SLICE만큼 실행되면, yield 한다.

이후 wake up으로 다른 thread를 실행시킨다.

aging을 사용할 경우 대기한 thread의 우선 순위를 올린다.

마지막으로 MLFQ의 경우 모든 thread 중 최고 우선 순위인 애를, 일반 priority의 경우 리스트의 맨 앞 thread의 우선 순위가 현재보다 높으면 양보 받도록 한다.

```

void thread_sleep (int64_t wakeup_tick) {
    struct thread *cur = thread_current ();
    enum intr_level old_level;

    ASSERT (!intr_context ());
    ASSERT (intr_get_level () == INTR_ON);

    cur->wakeup_tick = wakeup_tick;

    old_level = intr_disable ();
    list_push_back (&sleep_list, &cur->elem);
    thread_block ();

    intr_set_level (old_level);
}

```

```

void thread_wake_up (void) {
    int64_t current_ticks = timer_ticks ();
    struct list_elem *e = list_begin (&sleep_list);

    while (e != list_end (&sleep_list)) {
        struct thread *t = list_entry (e, struct thread, elem);

        if (t->wakeup_tick <= current_ticks) {
            struct list_elem *next = list_next (e);
            list_remove (e);
            thread_unblock (t);
            e = next;
        }
        else e = list_next (e);
    }
}

```

thread_sleep()은 현재 thread에게 깨어나는 데 필요한 시간을 부여하고 sleep 리스트에 삽입하고, thread_block()을 통해 재운다.

thread_wake_up()은 매 tick마다 sleep 리스트를 탐색하여 깨어날 시간이 된 thread를 꺼내서 thread_unblock()을 호출하여 ready 큐에 삽입한다.

```

void thread_aging (void) {
    ASSERT (intr_get_level () == INTR_OFF);

    struct list_elem *e;

    for (e = list_begin (&ready_list); e != list_end (&ready_list); e = list_next (e)) {
        struct thread *t = list_entry (e, struct thread, elem);
        if (t->base_priority < PRI_MAX) {
            t->base_priority++;
            thread_recalculate_priority (t);
        }
    }

    for (e = list_begin (&sleep_list); e != list_end (&sleep_list); e = list_next (e)) {
        struct thread *t = list_entry (e, struct thread, elem);
        if (t->base_priority < PRI_MAX) t->base_priority++;
    }

    list_sort (&ready_list, thread_priority_greater, NULL);
}

```

starvation 방지를 위해 대기 중인 thread의 우선 순위를 높인다. 매 tick마다 ready 큐와 sleep 리스트를 탐색하여 base priority < PRI_MAX 인 경우 base priority를 높인다. 마지막으로 ready 큐를 우선 순위 순으로 재정렬한다.

```

void mlfqs_update_load_avg (void) {
    int ready_threads = 0;

    for (int i = PRI_MIN; i <= PRI_MAX; i++) ready_threads += list_size (&ready_list[i]);

    if (thread_current () != idle_thread) ready_threads++;

    int term1 = FP_DIV_INT (FP_MUL_INT (load_avg, 59), 60);
    int term2 = FP_DIV_INT (INT_TO_FP (ready_threads), 60);
    load_avg = FP_ADD (term1, term2);
}

```

1초마다 시스템 전체의 load avg를 공식을 이용해 갱신한다.

```

void mlfqs_calculate_recent_cpu (struct thread *t) {
    if (t == idle_thread) return;

    int load_x_2 = FP_MUL_INT (load_avg, 2);
    int coeff = FP_DIV (load_x_2, FP_ADD_INT (load_x_2, 1));

    t->recent_cpu = FP_ADD_INT (FP_MUL (coeff, t->recent_cpu), t->nice);
}

```

1초마다 모든 thread의 recent cpu 값을 공식을 이용해 갱신한다.

```

void mlfqs_calculate_priority (struct thread *t) {
    if (t == idle_thread) return;

    int recent_cpu_div_4 = FP_TO_INT_TRUNC (FP_DIV_INT (t->recent_cpu, 4));
    int nice_x_2 = t->nice * 2;

    int priority = PRI_MAX - recent_cpu_div_4 - nice_x_2;

    if (priority < PRI_MIN) priority = PRI_MIN;
    if (priority > PRI_MAX) priority = PRI_MAX;

    t->priority = priority;
}

```

4 tick마다 모든 thread의 우선 순위를 계산한다. 이때도 앞서 기재한 공식을 이용한다.

```
void mlfqs_increment_recent_cpu (void) {
    struct thread *cur = thread_current ();
    if (cur == idle_thread) return;

    cur->recent_cpu = FP_ADD_INT (cur->recent_cpu, 1);
}
```

매 tick마다 현재 실행 중인 thread의 recent cpu를 1 증가한다.

```
void mlfqs_recalculate_all (void) {
    struct list_elem *e;
    bool is_second = (timer_ticks () % TIMER_FREQ == 0);
    bool is_4th_tick = (timer_ticks () % TIME_SLICE == 0);

    if (is_second) mlfqs_update_load_avg ();

    for (e = list_begin (&all_list); e != list_end (&all_list); e = list_next (e)) {
        struct thread *t = list_entry (e, struct thread, allelem);
        if (t == idle_thread) continue;

        if (is_second) mlfqs_calculate_recent_cpu (t);
        if (is_4th_tick) mlfqs_calculate_priority (t);
    }

    if (is_4th_tick) {
        for (int i = PRI_MIN; i <= PRI_MAX; i++) {
            struct list_elem *e = list_begin (&ready_list[i]);
            while (e != list_end (&ready_list[i])) {
                struct thread *t = list_entry (e, struct thread, elem);
                struct list_elem *next = list_next (e);
                if (t->priority != i) {
                    list_remove (e);
                    list_push_back (&ready_list[t->priority], &t->elem);
                }
                e = next;
            }
        }
    }
}
```

1초마다 시스템 전체의 load avg를 갱신한다. 매 tick마다 해당 함수가 호출되기에, 해당 함수에서 mlfqs_calculate_recent_cpu()와 mlfqs_calculate_priority()를 호출해서 모든 thread의 recent cpu와 우선 순위를 갱신한다. 그리고 매 4tick마다 모든 thread의 우선 순위를 다시 계산한다.

- 개발 중 발생한 문제나 이슈가 있으면 이를 간략히 설명하고 해결한 방식에 대해 설명

모든 함수에서 ready 큐 또는 sleep 리스트를 다룰 때 empty 확인을 하지 않으면 오류가 발생하는 것을 확인했다. 함수 앞부분에 empty check를 함으로써 방지할 수 있었다.

Pintos의 환경에선 부동 소수점 지원이 되지 않는 이슈가 있었다. 직접 부동 소수점과 관련된 연산 함수들을 새로운 헤더 파일에 define하여 이를 해결했다.

C. 시험 및 평가 내용

- priority-lifo.c 코드 및 priority-lifo 테스트 결과 분석

```
Boot complete.
Executing 'priority-lifo':
(priority-lifo) begin
(priority-lifo) 16 threads will iterate 16 times in the same order each time.
(priority-lifo) If the order varies then there is a bug.
(priority-lifo) iteration: 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15
(priority-lifo) iteration: 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14
(priority-lifo) iteration: 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13
(priority-lifo) iteration: 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12
(priority-lifo) iteration: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11
(priority-lifo) iteration: 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
(priority-lifo) iteration: 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
(priority-lifo) iteration: 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
(priority-lifo) iteration: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
(priority-lifo) iteration: 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
(priority-lifo) iteration: 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
(priority-lifo) iteration: 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
(priority-lifo) iteration: 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
(priority-lifo) iteration: 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
(priority-lifo) iteration: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
(priority-lifo) iteration: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(priority-lifo) end
Execution of 'priority-lifo' complete.
Timer: 30 ticks
Thread: 0 idle ticks, 30 kernel ticks, 0 user ticks
Console: 1557 characters output
Keyboard: 0 keys pressed
Powering off...
cse20211558@cspro5:~/pintos/src/threads/build$
```

메인 thread가 자신의 id출력하는 것을 총 16회 실행한 후 다음 thread에게 yield한다. 이때 priority 순서대로 15부터 0번의 thread들이 실행된다.

- make check 수행 결과를 캡처하여 첨부

```
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-change-2
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-aging
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
```

모든 test case를 pass한 것을 확인할 수 있다.